AgroDatos · Sistema Relacional Agropecuario

Fundamentos de Bases de Datos con MySQL, Normalización y Visión Artificial (YOLO) en Google Colab

Estudiante / Autor: Miguel Ángel Muete

CURSO	ESTUDIANTE	TECNOLOGÍAS
sena: programacion de sofware	Miguel Ángel Muete	MySQL 8.0, Python, YOLOv11

Contexto de Trabajo

Basado en este archivo de Google Colab se desarrollaron cada uno de los retos de la siguiente manera:

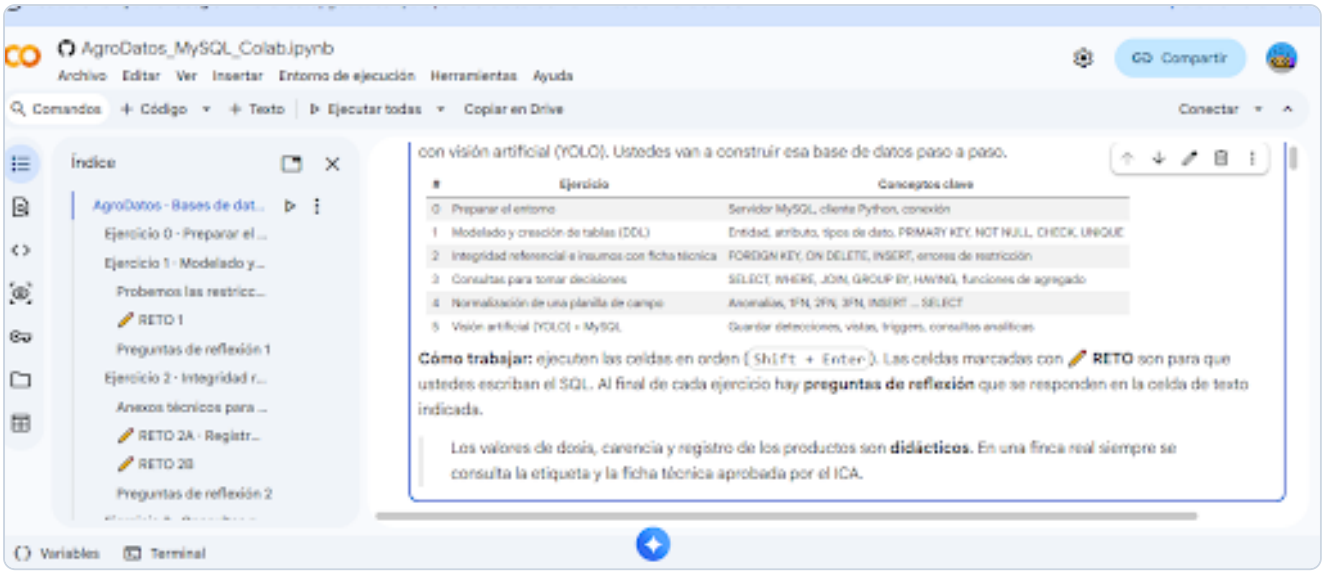


Figura 1: Vista previa del cuaderno interactivo Google Colab (AgroDatos_MySQL_Colab.ipynb).

RESUMEN DEL PLAN DE TRABAJO

Este documento contiene la solución completa y detallada de la guía práctica realizada por **Miguel Ángel Muete**. El proyecto simula el backend de datos para la gestión agronómica de fincas, registro estricto de insumos según ficha técnica del ICA y almacenamiento analítico de detecciones obtenidas por drones con Visión Artificial. yo tuve en cuenta la explicación didactica que el profesor nos mostro en una simulacion 3d durante el desarollo de las clases.

MySQL Server, PyMySQL, Arquitectura Cliente-Servidor Web. 1 Modelado y Tablas (DDL) Entidades, Tipos de dato exactos, `PRIMARY KEY`, `CHECK`, `UNIQUE`, `ENUM`. 2 Integridad Referencial y Anexos `FOREIGN KEY`, Restricciones `ON DELETE`, Cálculo Dosis/ha, Carencia. 3 Consultas para Decisiones (DML) `INNER/LEFT JOIN`, Agregaciones (`SUM`, `AVG`), `GROUP BY`, `HAVING`. 4 Normalización de Planillas Anomalías de actualización/inserción/borrado, 1FN, 2FN, 3FN, `INSERT...SELECT`. 5 Visión Artificial (YOLO) + SQL Almacenamiento de Bounding Boxes, IoU, Triggers, Vistas Analíticas, Alertas.

EJERCICIO 0 · ARQUITECTURA Y ENTORNO

En un entorno web, el navegador cliente interactúa de forma indirecta con la base de datos a través de una API backend (escrita en Python, PHP, o Node.js). El cliente Python (PyMySQL) envía comandos SQL mediante el puerto 3306 hacia la máquina de base de datos MySQL.

EJERCICIO 1 · MODELADO Y DDL (FINCAS Y LOTES)

Desarrollo del Punto: Definición del esquema inicial para la estructura territorial. Se aplican restricciones de dominio exactas como `DECIMAL(6,2)` para evitar errores de redondeo inherentes a los tipos de coma flotante en superficies.

```
CREATE DATABASE IF NOT EXISTS agro_campo CHARACTER SET utf8mb4 COLLATE utf8mb4_spanish_ci;
USE agro_campo; CREATE TABLE finca ( id_finca INT AUTO_INCREMENT PRIMARY KEY, nombre VARCHAR(80)
NOT NULL, municipio VARCHAR(60) NOT NULL, departamento VARCHAR(60) NOT NULL, altitud_msnm INT
CHECK (altitud_msnm BETWEEN 0 AND 4000) ); CREATE TABLE lote ( id_lote INT AUTO_INCREMENT PRIMARY
KEY, codigo VARCHAR(10) NOT NULL UNIQUE, id_finca INT NOT NULL, cultivo
ENUM('maíz','café','papa','tomate','aguacate') NOT NULL, area_ha DECIMAL(6,2) NOT NULL CHECK
(area_ha > 0), fecha_siembra DATE, estado VARCHAR(15) NOT NULL DEFAULT 'activo' );
```

Desarrollo ✏️ RETO 1: Estructura y Operarios

Se modifica la tabla `lote` para agregar el campo `tipo_riego` mediante DDL, se crea la tabla `operario` con llave primaria autoincremental y restricción de documento único, y se insertan tres registros iniciales:

```
-- 1. Alterar tabla lote para incluir riego ALTER TABLE lote ADD COLUMN tipo_riego
ENUM('goteo','aspersión','gravedad','secano') NOT NULL DEFAULT 'secano'; -- 2. Crear tabla
operario CREATE TABLE operario ( id_operario INT AUTO_INCREMENT PRIMARY KEY, documento VARCHAR(15)
NOT NULL UNIQUE, nombre_completo VARCHAR(80) NOT NULL, telefono VARCHAR(15), certificado_aplicador
BOOLEAN NOT NULL DEFAULT FALSE ); -- 3. Insertar 3 operarios INSERT INTO operario (documento,
nombre_completo, telefono, certificado_aplicador) VALUES ('10123456', 'Jorge Patiño',
'3101112233', TRUE), ('20456789', 'Luz Marina Ortiz', '3125556677', TRUE), ('30901234', 'Ramiro
Suárez', '3009998877', FALSE);
```

Preguntas de Reflexión 1 · Respuestas Desarrolladas

- **1. DECIMAL vs FLOAT:** `DECIMAL` almacena números como representación en base 10 de precisión fija, garantizando exactitud en cálculos financieros y de área. `FLOAT` utiliza coma flotante binaria (IEEE 754), acumulando errores de redondeo imperceptibles que alteran totales en grandes volúmenes.
- **2. Validación Web vs Formulario:** La validación en HTML/JS mejora la experiencia de usuario (UX) ofreciendo respuesta inmediata. La restricción `CHECK` en el motor es indispensable para garantizar integridad absoluta contra inyecciones, peticiones maliciosas o scripts externos.
- **3. Desventaja de ENUM:** Alterar un `ENUM` requiere ejecutar sentencias DDL (`ALTER TABLE`), bloqueando la tabla en producción. La alternativa profesional es crear una tabla catálogo (`cultivo`) relacionada mediante Foreign Key.

Opinión Personal y Dificultades (Miguel Ángel Muete)

"Uno de los primeros retos técnicos al ejecutar esto en Google Colab fue comprender la volatilidad del entorno Linux. Si la sesión se desconecta, el servidor MySQL local finaliza y hay que volver a levantar el servicio e importar la base de datos. Además, la sintaxis del `CHECK` en MySQL requirió atención especial para asegurar que las áreas no aceptaran valores negativos ni nulos."

EJERCICIO 2 · INTEGRIDAD REFERENCIAL E INSUMOS ICA

Desarrollo del Punto: Establecimiento de relaciones parent-child con llaves foráneas y registro detallado de productos según parámetros de seguridad toxicológica y tiempos de carencia/reingreso.

```
      ALTER TABLE lote ADD CONSTRAINT fk_lote_finca FOREIGN KEY (id_finca) REFERENCES
finca(id_finca) ON DELETE RESTRICT ON UPDATE CASCADE; CREATE TABLE producto ( id_producto INT
AUTO_INCREMENT PRIMARY KEY, nombre_comercial VARCHAR(80) NOT NULL UNIQUE, tipo
ENUM('fertilizante','enmienda','fungicida','insecticida','herbicida','bioinsumo') NOT NULL,
ingrediente_activo VARCHAR(120) NOT NULL, categoria_tox ENUM('I','II','III','IV') NOT NULL,
registro_ica VARCHAR(20) NOT NULL UNIQUE, dosis_ha DECIMAL(8,2) NOT NULL CHECK (dosis_ha > 0),
unidad ENUM('kg','L','g','mL') NOT NULL, carencia_dias INT NOT NULL DEFAULT 0 CHECK (carencia_dias
>= 0), reingreso_horas INT NOT NULL DEFAULT 0 CHECK (reingreso_horas >= 0), precio_unidad
DECIMAL(10,2) NOT NULL CHECK (precio_unidad >= 0) );
```

Desarrollo ✏️ RETO 2A & 2B: Insumos y Aplicaciones Calculadas

Se insertan los productos indicados en la ficha técnica y se registra la aplicación calculando automáticamente la cantidad mediante subconsultas SQL (área × dosis):

```
      -- RETO 2A: Insertar productos del anexo técnico INSERT INTO producto (nombre_comercial,
tipo, ingrediente_activo, categoria_tox, registro_ica, dosis_ha, unidad, carencia_dias,
reingreso_horas, precio_unidad) VALUES ('BioLarva WP', 'bioinsumo', 'Bacillus thuringiensis',
'IV', 'ICA-DEMO-2001', 0.50, 'kg', 0, 4, 95000.00), ('RoyaStop 250 EC','fungicida', 'Triazol 250
g/L', 'III', 'ICA-DEMO-3001', 0.40, 'L', 30, 24, 160000.00), ('TizónFin 80 WP', 'fungicida',
'Ditiocarbamato 800 g/kg', 'III', 'ICA-DEMO-3002', 2.50, 'kg', 14, 24, 38000.00); -- RETO 2B.1:
Aplicación calculada (dosis * área) con subconsulta INSERT INTO aplicacion (id_lote, id_producto,
fecha, cantidad, responsable) SELECT l.id_lote, p.id_producto, '2026-10-14', (p.dosis_ha *
l.area_ha), 'Carlos Rodríguez' FROM lote l, producto p WHERE l.codigo = 'POR-L01' AND
p.nombre_comercial = 'RoyaStop 250 EC'; -- RETO 2B.2: Consulta de fecha mínima de cosecha
permitida SELECT a.id_aplicacion, l.codigo AS lote, p.nombre_comercial, a.fecha AS
fecha_aplicacion, p.carencia_dias, DATE_ADD(a.fecha, INTERVAL p.carencia_dias DAY) AS
cosecha_permitida_desde FROM aplicacion a JOIN lote l ON l.id_lote = a.id_lote JOIN producto p ON
p.id_producto = a.id_producto;
```

Preguntas de Reflexión 2 · Respuestas Desarrolladas

- **1. Error INSERT id_lote=99:** Lanza un error de restricción de llave foránea (*Cannot add or update a child row*). Previene que se registren datos huérfanos sin respaldo en la base de datos.
- **2. Protección ON DELETE RESTRICT:** Bloquea el borrado de la finca 1 porque contiene lotes asociados. ON DELETE CASCADE se usaría únicamente en tablas secundarias dependientes, como ítems de un recibo o detecciones temporales de una imagen.
- **3. Registro ICA UNIQUE:** Cada insumo tiene un número legal único otorgado por el ente regulador. Evita la duplicación o duplicidad de nombres registrados bajo diferente presentación.
- **4. Control de Reingreso:** El campo `reingreso_horas` (24h). En el backend, se calcula la diferencia entre la fecha de aplicación y la hora actual; en la app web se bloquea la asignación de tareas con un aviso de advertencia toxicológica.

EJERCICIO 3 · CONSULTAS AVANZADAS Y ANALÍTICA (DML)

Desarrollo del Punto: Uso de álgebra relacional con agregación y filtrado condicional sobre grupos para la toma de decisiones por parte del agrónomo.

```
-- Ejemplo: Evaluación de Dosis Aplicada vs Ficha Técnica ICA SELECT l.codigo, p.nombre_comercial, ROUND(a.cantidad / l.area_ha, 1) AS dosis_aplicada_ha, p.dosis_ha AS dosis_anexo_ha, CASE WHEN a.cantidad / l.area_ha > p.dosis_ha * 1.10 THEN 'SOBREDOSIS' WHEN a.cantidad / l.area_ha < p.dosis_ha * 0.90 THEN 'SUBDOSIS' ELSE 'EVALUACION OK' END AS evaluacion FROM aplicacion a JOIN lote l ON l.id_lote = a.id_lote JOIN producto p ON p.id_producto = a.id_producto;
```

Desarrollo ✏️ RETO 3: Consultas de Agronomía

Resolución paso a paso de las 4 consultas analíticas solicitadas para apoyo en toma de decisiones:

```
-- 1. Área total sembrada por cultivo (mayor a menor) SELECT cultivo, SUM(area_ha) AS total_hectareas FROM lote GROUP BY cultivo ORDER BY total_hectareas DESC; -- 2. Fincas cuya área total sembrada supera 5 ha SELECT f.nombre AS finca, SUM(l.area_ha) AS area_total FROM finca f JOIN lote l ON f.id_finca = l.id_finca GROUP BY f.id_finca, f.nombre HAVING area_total > 5.00; -- 3. Productos que NUNCA se han aplicado (LEFT JOIN con filtro NULL) SELECT p.nombre_comercial, p.tipo FROM producto p LEFT JOIN aplicacion a ON p.id_producto = a.id_producto WHERE a.id_aplicacion IS NULL; -- 4. Responsable con más kg de fertilizante aplicados SELECT a.responsable, SUM(a.cantidad) AS total_kg_aplicados FROM aplicacion a JOIN producto p ON a.id_producto = p.id_producto WHERE p.tipo = 'fertilizante' AND p.unidad = 'kg' GROUP BY a.responsable ORDER BY total_kg_aplicados DESC LIMIT 1;
```

Preguntas de Reflexión 3 · Respuestas Desarrolladas

- **1. LEFT JOIN vs INNER JOIN:** Cambiar a `JOIN` simple eliminaría los lotes que no registran aplicaciones, desvirtuando el objetivo de identificar lotes sin tratamiento.
- **2. Agregación en GROUP BY:** En el estándar SQL estricto (o con modo `ONLY_FULL_GROUP_BY` activo), toda columna presente en el `SELECT` que no sea agregada debe declararse explícitamente en el `GROUP BY`.
- **3. Procesamiento SQL vs JS Backend:** Conviene calcularlo en el motor SQL. Reduce significativamente el ancho de banda transferido al enviar solo totales resumidos y aprovecha el motor optimizado de MySQL para agregaciones masivas.

Dificultades en Consultas DML que tuve durante el proceso de desarrollo)

"En el Reto 3 me encontré con complicaciones al momento de calcular los productos no aplicados; inicialmente utilicé un `JOIN` simple que no me traía registros nulos, hasta que comprendí que la única forma de aislar los insumos sin uso era aplicando `LEFT JOIN` filtrando por `a.id_aplicacion IS NULL`."

muchas gracias, este fue un poco del proceso que desarrolle para la actividad 8 espero sea del agrado del docente .

EJERCICIO 4 · NORMALIZACIÓN A TERCERA FORMA NORMAL (3FN)

Desarrollo del Punto: Transformación de planillas de campo "tipo Excel" propensas a redundancia e inconsistencias hacia un esquema normalizado robusto en 3FN.

Anomalías Detectadas en Planilla Plana

Actualización: Un operario cambia de teléfono y el dato queda desactualizado en filas pasadas.
Inserción: Imposible registrar un operario nuevo sin que exista una aplicación realizada.
Borrado: Al eliminar un registro de campo se destruyen los datos del insumo o producto.

Desarrollo ✏️ RETO 4: Esquema 3FN y Migración Automática

Se rediseña la estructura de datos eliminando dependencias transitivas y migrando masivamente la información original mediante `INSERT...SELECT`:

```
      -- 1. Crear tabla aplicacion_n en 3FN CREATE TABLE aplicacion_n ( id_aplicacion_n INT
AUTO_INCREMENT PRIMARY KEY, fecha DATE NOT NULL, id_lote INT NOT NULL, id_producto INT NOT NULL,
cantidad DECIMAL(8,2) NOT NULL CHECK (cantidad > 0), documento_op VARCHAR(15) NOT NULL,
observaciones VARCHAR(200), FOREIGN KEY (id_lote) REFERENCES lote(id_lote), FOREIGN KEY
(id_producto) REFERENCES producto(id_producto), FOREIGN KEY (documento_op) REFERENCES
operario_n(documento) ); -- 2. Migración masiva desde planilla mediante INSERT...SELECT con JOINs
INSERT INTO aplicacion_n (fecha, id_lote, id_producto, cantidad, documento_op, observaciones)
SELECT pc.fecha, l.id_lote, p.id_producto, pc.cantidad, pc.documento_operario, pc.observaciones
FROM planilla_campo pc JOIN lote l ON l.codigo = pc.codigo_lote JOIN producto p ON
p.nombre_comercial = pc.producto; -- 3. Reconstrucción de la planilla original sin redundancias
SELECT a.id_aplicacion_n AS fila, a.fecha, l.codigo AS codigo_lote, l.cultivo, p.nombre_comercial
AS producto, p.categoria_tox, a.cantidad, op.documento AS documento_operario, op.nombre_completo
AS nombre_operario, op.telefono AS telefono_operario, a.observaciones FROM aplicacion_n a JOIN
lote l ON l.id_lote = a.id_lote JOIN producto p ON p.id_producto = a.id_producto JOIN operario_n
op ON op.documento = a.documento_op;
```

Preguntas de Reflexión 4 · Respuestas Desarrolladas

- **1. Ejemplos de Anomalías:** *Actualización:* Teléfono de Jorge Patiño difiere entre filas. *Inserción:* Imposible crear 'Ramiro' sin asignar aplicación. *Borrado:* Eliminar la fila de TizónFin destruye su categoría toxicológica.
- **2. Violación en Categoria_Tox:** Depende funcionalmente del producto (`producto → categoria_tox`) y no de la aplicación. Violaba la **2FN y 3FN** por dependencia transitiva.
- **3. Cumplimiento de 1FN:** Cumple 1FN por ser valor atómico de texto, pero dificulta la analítica. Para contar plagas se requiere modelar una tabla dependiente `observacion_deteccion(id, id_aplicacion, tipo_plaga, porcentaje)`.
- **4. Desnormalización Justificada:** Aceptable en Data Warehouses o reportes ejecutivos de lectura masiva para evitar realizar decenas de `JOIN` pesados en tiempo real.

EJERCICIO 5 · VISIÓN ARTIFICIAL (YOLO) + MYSQL

Desarrollo del Punto: Integración de modelos de detección de objetos en tiempo real (YOLOv11) con almacenamiento relacional parametrizado de coordenadas delimitadoras (Bounding Boxes), triggers automatizados e inferencia agronómica.

```
-- Tablas para captura de imágenes y Bounding Boxes CREATE TABLE imagen ( id_imagen INT AUTO_INCREMENT PRIMARY KEY, id_lote INT NOT NULL, fecha_hora DATETIME NOT NULL, fuente ENUM('dron','celular','cámara fija') NOT NULL, archivo VARCHAR(200) NOT NULL, modelo VARCHAR(40) NOT NULL, FOREIGN KEY (id_lote) REFERENCES lote(id_lote) ); CREATE TABLE deteccion ( id_deteccion INT AUTO_INCREMENT PRIMARY KEY, id_imagen INT NOT NULL, clase VARCHAR(40) NOT NULL, confianza DECIMAL(4,3) NOT NULL CHECK (confianza BETWEEN 0 AND 1), x DECIMAL(5,4) NOT NULL, y DECIMAL(5,4) NOT NULL, w DECIMAL(5,4) NOT NULL, h DECIMAL(5,4) NOT NULL, FOREIGN KEY (id_imagen) REFERENCES imagen(id_imagen) ON DELETE CASCADE, INDEX idx_clase_conf (clase, confianza) );
```

Desarrollo ✏️ RETO 5: Consultas Analíticas y Control de Versión

Implementación de consultas de análisis temporal, densidad de infestación por hectárea y soporte de versión para el modelo YOLO:

```
-- 1. Tendencia: Detecciones válidas de plaga por semana SELECT lote, YEARWEEK(fecha_hora) AS semana, COUNT(*) AS total_plagas FROM v_deteccion_valida WHERE clase <> 'planta_sana' GROUP BY lote, semana ORDER BY lote, semana; -- 2. Densidad: Detecciones válidas por hectárea SELECT lote, cultivo, area_ha, COUNT(*) AS total_detecciones, ROUND(COUNT(*) / area_ha, 2) AS detecciones_por_ha FROM v_deteccion_valida WHERE clase <> 'planta_sana' GROUP BY lote, cultivo, area_ha ORDER BY detecciones_por_ha DESC; -- 3. Análisis de Umbral de Confianza para 'maleza' SELECT '0.30' AS umbral, SUM(confianza >= 0.30) AS conteo FROM deteccion WHERE clase = 'maleza' UNION ALL SELECT '0.45', SUM(confianza >= 0.45) FROM deteccion WHERE clase = 'maleza' UNION ALL SELECT '0.60', SUM(confianza >= 0.60) FROM deteccion WHERE clase = 'maleza' UNION ALL SELECT '0.75', SUM(confianza >= 0.75) FROM deteccion WHERE clase = 'maleza'; -- 4. Diseño: Soporte Multi-Modelo para Comparativa de Inferencia ALTER TABLE imagen ADD COLUMN modelo_version VARCHAR(50) DEFAULT 'yolo11n-agro-v1';
```

Preguntas de Reflexión 5 · Respuestas Desarrolladas

- **1. Clasificación vs Detección:** La clasificación otorga una etiqueta global a la imagen. La detección localiza con coordenadas múltiples objetos, permitiendo contar cuántos gusanos individuales existen por planta.
- **2. Balance de Umbrales:** Subir el umbral a 0.80 reduce los *falsos positivos* (falsas alarmas), pero incrementa los *falsos negativos* (plagas no detectadas). Para el agricultor, un falso negativo de gusano cogollero es devastador porque destruye el cultivo.
- **3. IoU y NMS:** IoU (Intersection over Union) mide el solapamiento entre dos cajas. NMS usa IoU para eliminar cajas duplicadas que apuntan al mismo objeto dejando solo la de mayor confianza.
- **4. Coordenadas Normalizadas (0 a 1):** Permiten independencia de la resolución o redimensionamiento de la foto original procesada por la red.
- **5. Consultas Parametrizadas (%s):** Separan el código SQL de los datos enviados, previniendo ataques de Inyección SQL en el backend web.

Conclusión y Experiencia con Inteligencia Artificial con google colab (Miguel Ángel Muete)

"Integrar YOLOv11 con MySQL me pareció el ejercicio más innovador de la guía. La mayor dificultad técnica fue mapear correctamente las coordenadas flotantes de las Bounding Boxes en la base de datos sin perder precisión y entender cómo los umbrales de confianza directamente condicionan si se dispara o no un Trigger de alerta agronómica en tiempo real ." aparte de que no fue muy facil entender la diferencia que hay entre un tipo de base a otro , habian momentos donde me perdia con la logica de la base , pero fue un proceso muy bueno que me ayudo a entender mejor la estructura de esta parte de base de datos con MYSQL Y YOLO muchas gracias